GOMYCODE

Build your AI Assistant with LLMs & RAG

4-HOUR WORKSHOP

TABLE OF CONTENTS

Introducing GOMYCODE

Who are we?

GOMYCODE is an innovative educational Startup that trains top talents on the latest digital skills through a hybrid learning model that combines in-person guidance and an online learning platform.

we want to help YOU learn new skills

GOMYCODE©

1. En ligne
2. El Mourouj
3. Bardo
4. Boumhel
5. Lac 1
6. El Menzah
7. Nabeul
8. Sousse
9. Kairouan
10. Gabes
11. Tozeur
12. Tataouine

Specialities

Artificial Intelligence

Cloud, DevOps and Cybersecurity

Software Development

Data

Marketing

Design

We found **30** courses available for you

Data Scientist Bootcamp

Download the brochure
Acquire New Skills with...

AWS Cloud Practitioner...

See the detailed program
Learn Alongside Passionate...

Azure Fundamentals Certification...

See the detailed program
Learn Alongside Passionate...

Generative AI for Professionals

See the detailed program You want to learn more about ...

The DevOps Bootcamp: Kubernet...

See the detailed program
Learn Alongside Passionate...

Build your Product with AI tools

See the detailed program
Boost Your Skills with...

GenAI and LLMs – NVIDIA Certified

See the detailed program
Learning with Passionate...

Introduction to Artificial Intelligence

If you are into the field of artificial intelligence, this ...

Join a community of leaders and learners

Fill out the form below to receive the program that you're interested in.

Gomycode.com

First Name *

Last Name *

Phone *

Email *

Hackerspaces * Select an option

Product * Select an option

Submit Reset

Foundational Knowledge

Foundational Knowledge ‘AI’

[What is Artificial Intelligence (AI) ?]

Is all about making computers and machines smart. It’s like teaching a computer to think, learn, and solve problems, just like a human.

[Everyday examples of AI]

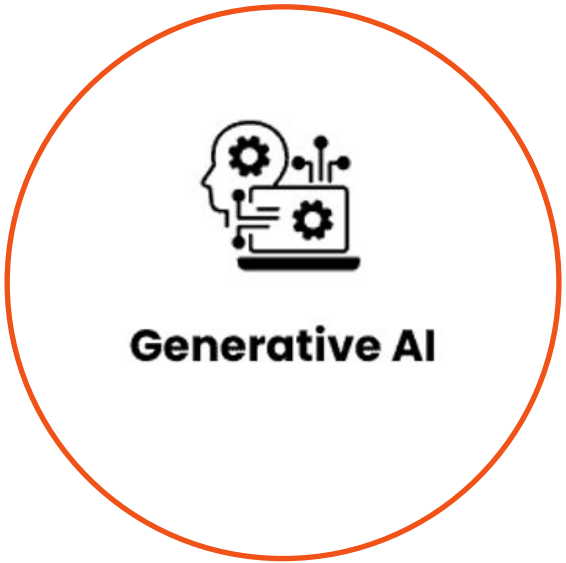
- chatbots (Answering customer questions)

- Self-driving cars

- Face recognition on smartphones

Foundational Knowledge ‘AI’

[Generative AI?]



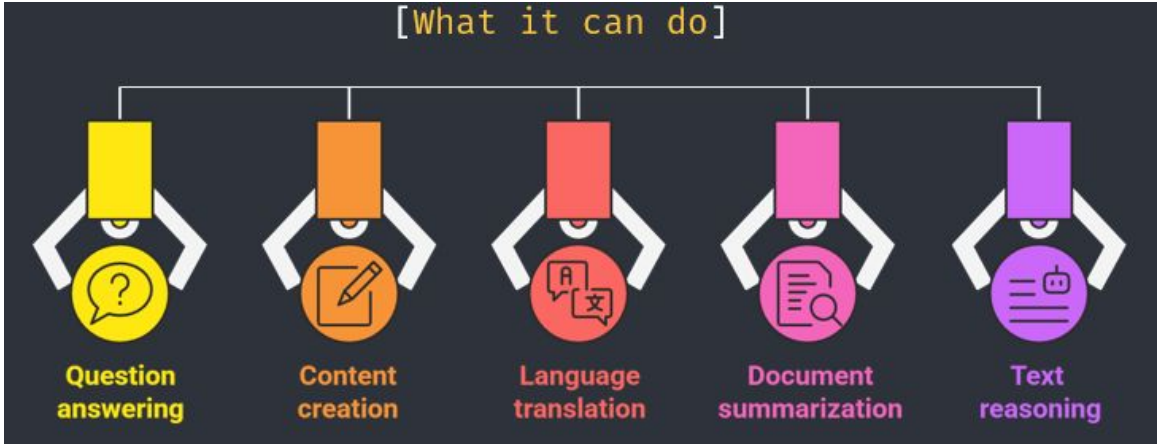
- Generative AI is a type of artificial intelligence that can create new content, rather than just analyzing or classifying existing data.
- it’s about generating something completely new. It can create text, images, music, and even videos.

Foundational Knowledge ‘LLM’

[LLM (Large Language Model)]

A model trained primarily on text to understand and generate language.

Input: 📝 Text → **Output:** 📝 Text



Foundational Knowledge

‘Prompt’

A prompt is a short piece of text that you give to the AI to tell it what you want it to create.

more details → better result

Example:

Beginner Prompt
“Write an email to a client.”

Good Prompt
“Write a professional email to a client named Sarah explaining that the project report will be delivered on Friday instead of Wednesday.”

AI Chatbot with LLM

Requirements

AI’s Weakness

Prompt: Write me a verse of poetry in Arabic by Abu al-Qasim al-Shabbi that he said to the rapper Balti, and don't add anything else.

Hallucination

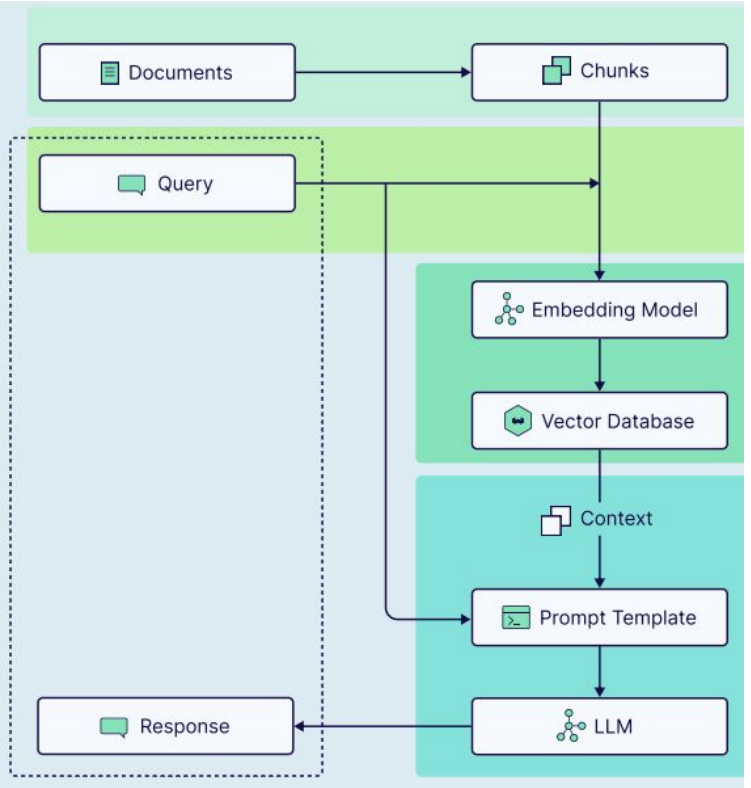
AI creating incorrect or made-up information

AI Chatbot with LLMs & RAG

Solution

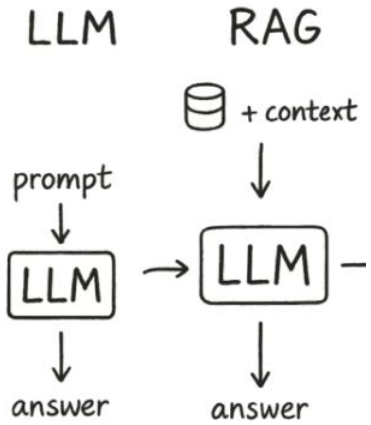
RAG : Retrieval augmented generation

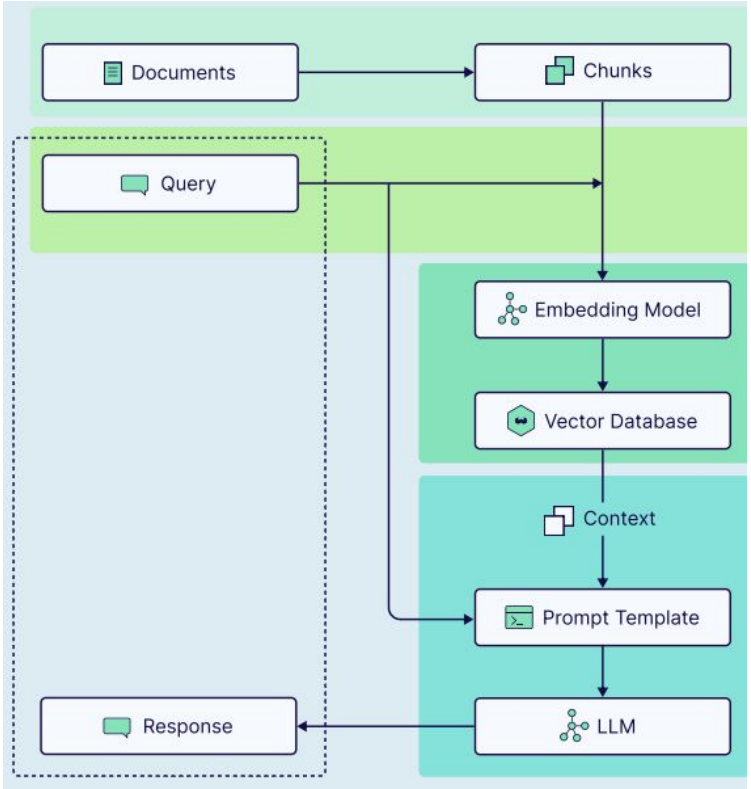
Is a technique that improves generative language model outputs by giving them access to an external source of knowledge.



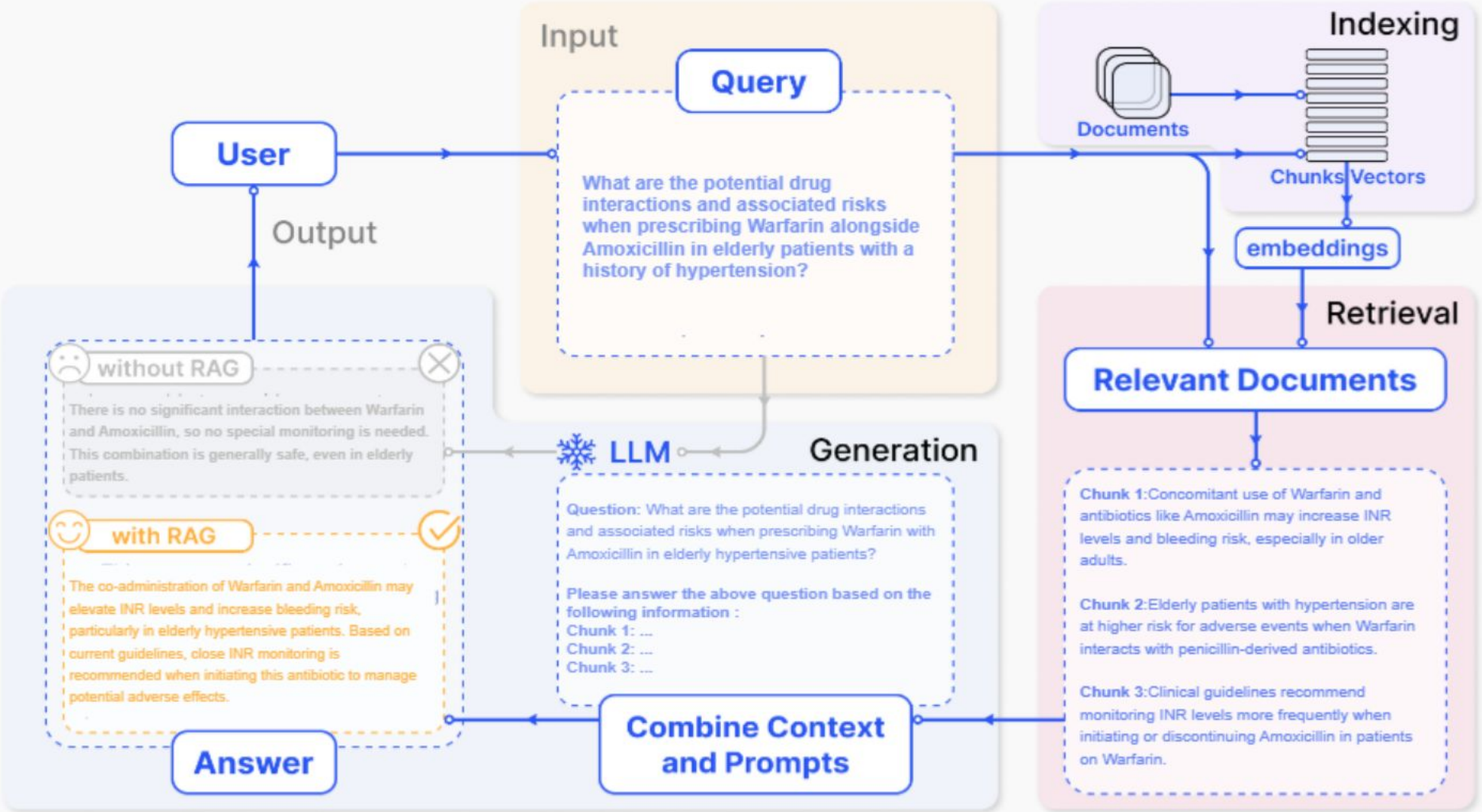
RAG : Retrieval augmented generation

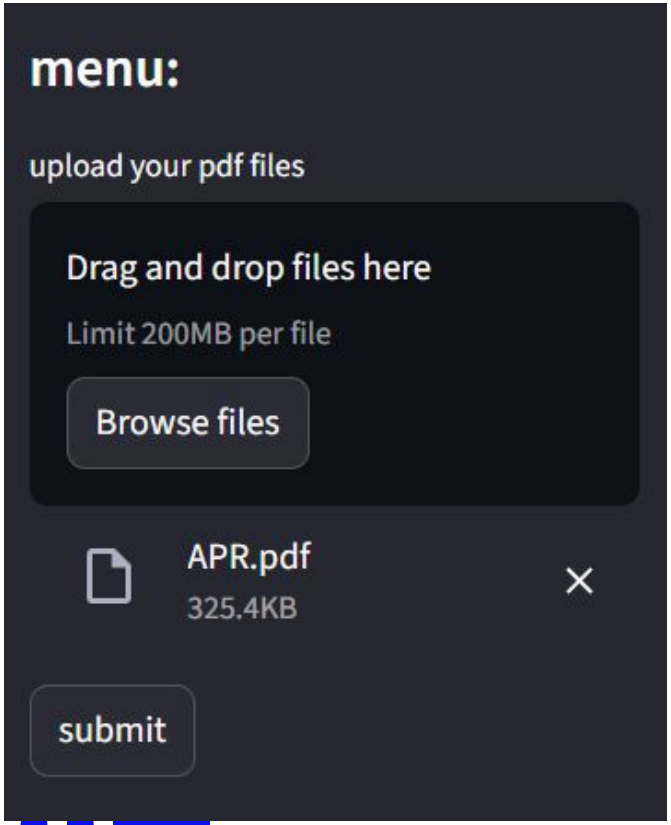
Unlike traditional language models that rely solely on training to generate responses, a RAG system first retrieves relevant information before producing a response.

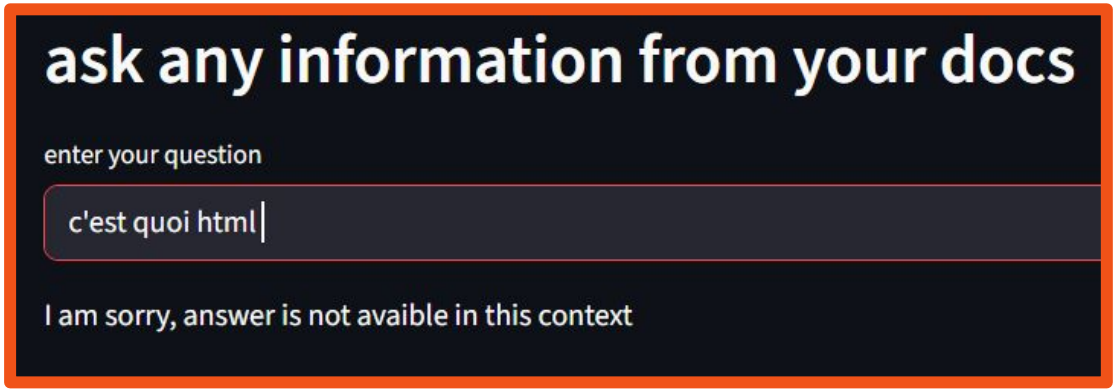


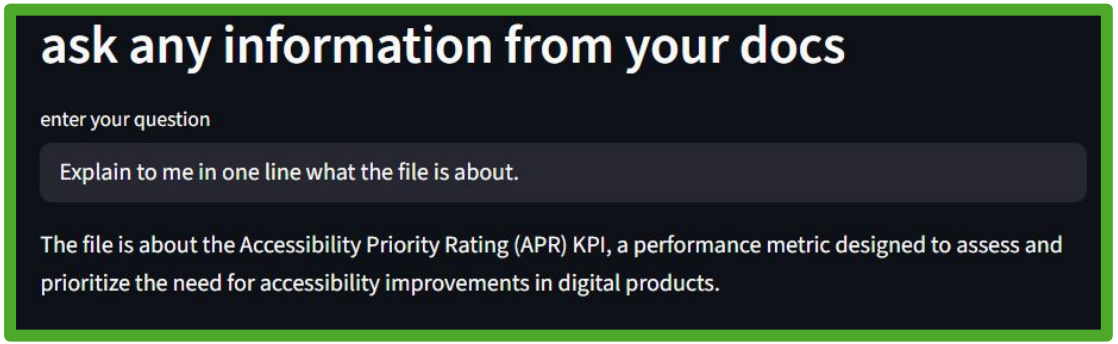


This reduces hallucinations and improves the accuracy of responses.


Input
Query
What are the potential drug interactions and associated risks when prescribing Warfarin alongside Amoxicillin in elderly patients with a history of hypertension?
User
Output
Indexing
Documents
Chunks Vectors
embeddings
Retrieval
Relevant Documents
Chunk 1:Concomitant use of Warfarin and antibiotics like Amoxicillin may increase INR levels and bleeding risk, especially in older adults.
Chunk 2:Elderly patients with hypertension are at higher risk for adverse events when Warfarin interacts with penicillin-derived antibiotics.
Chunk 3:Clinical guidelines recommend monitoring INR levels more frequently when initiating or discontinuing Amoxicillin in patients on Warfarin.
Combine Context and Prompts
LLM
Generation
Question: What are the potential drug interactions and associated risks when prescribing Warfarin with Amoxicillin in elderly hypertensive patients?
Please answer the above question based on the following information :
Chunk 1: ...
Chunk 2: ...
Chunk 3: ...
without RAG
There is no significant interaction between Warfarin and Amoxicillin, so no special monitoring is needed. This combination is generally safe, even in elderly patients.
with RAG
The co-administration of Warfarin and Amoxicillin may elevate INR levels and increase bleeding risk, particularly in elderly hypertensive patients. Based on current guidelines, close INR monitoring is recommended when initiating this antibiotic to manage potential adverse effects.
Answer

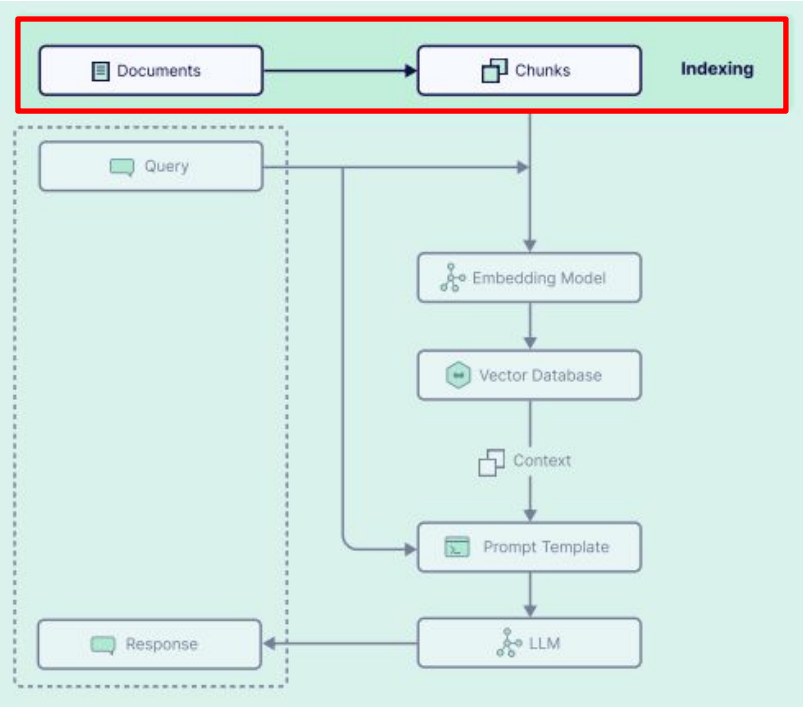

menu:
upload your pdf files
Drag and drop files here
Limit 200MB per file
Browse files
APR.pdf
325.4KB
submit


ask any information from your docs
enter your question
c'est quoi html
I am sorry, answer is not avaible in this context


ask any information from your docs
enter your question
Explain to me in one line what the file is about.
The file is about the Accessibility Priority Rating (APR) KPI, a performance metric designed to assess and prioritize the need for accessibility improvements in digital products.

Chunking Phase (Document Cutting)

Before a RAG system can search for information in a vector database, it must first prepare and organize documents into an exploitable form.



This preparation is based on two essential steps:

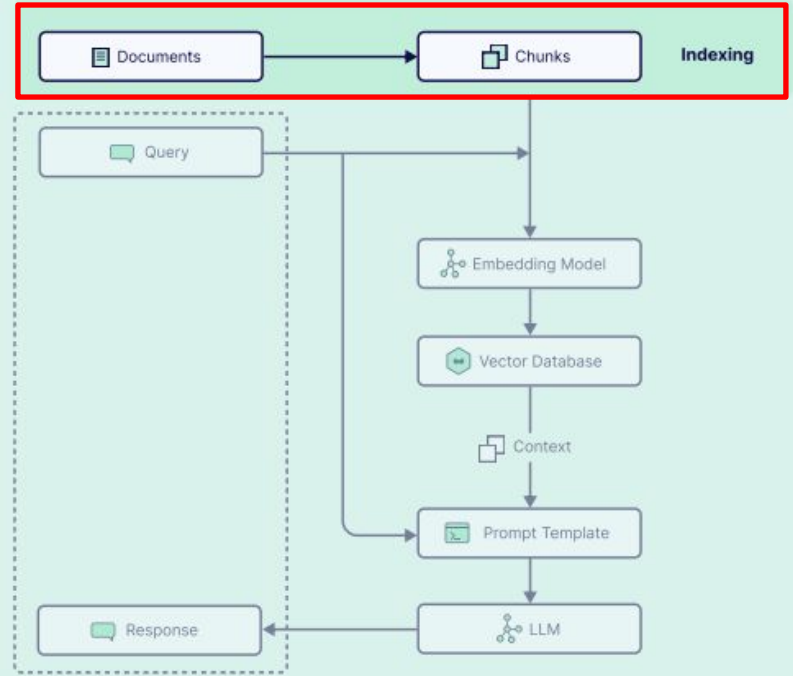
1- Chunking (cutting out documents)
2- The indexing of embeddings in the vector database

Chunking Phase (Document Cutting)

Chunking consists of cutting a document into small segments called chunks.
Each chunk represents a portion of text that can be processed and retrieved independently.

📌 Why do chunking?

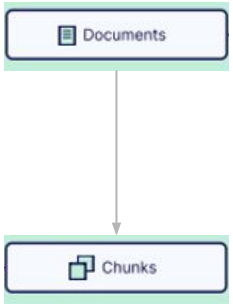
LLMs have a token limit (e.g. 4096, 8192 or more depending on the model).
Long documents cannot be processed at one time.
The small pieces allow better retrieval of information.



Coding

Chunking (cutting out documents)

```
def get_pdf_text(pdf_docs):
    text=""
    for pdf in pdf_docs:
        pdf_reader = PdfReader(pdf)
        for page in pdf_reader.pages:
            text+=page.extract_text()
    return text
```

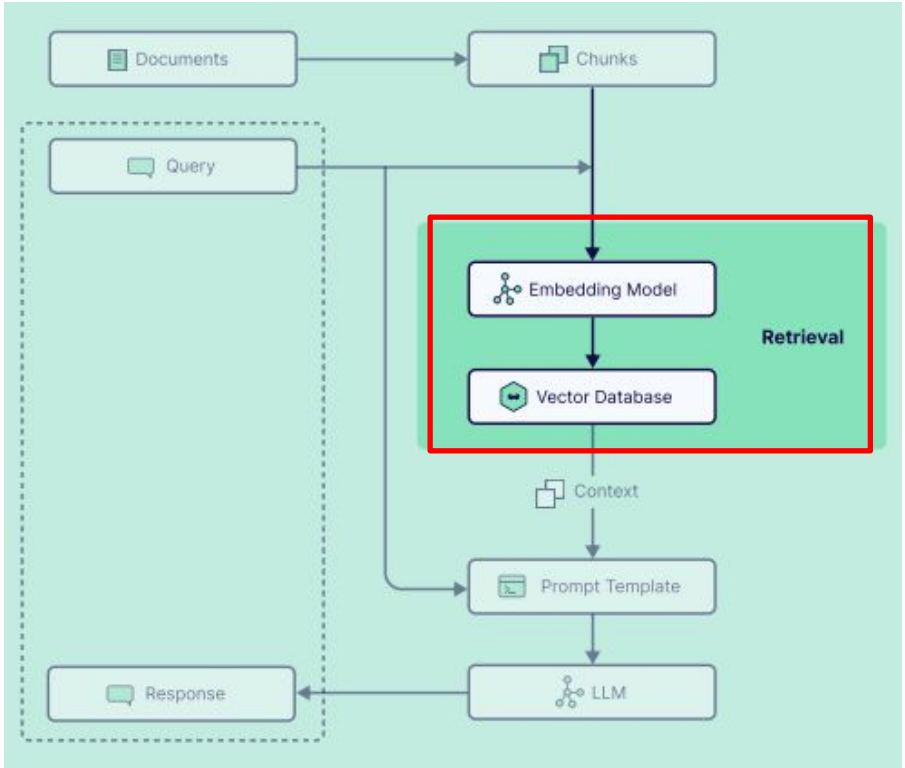



```
def get_text_chunks(text):
    text_splitter = RecursiveCharacterTextSplitter(chunk_size=5000, chunk_overlap=200)
    chunks = text_splitter.split_text(text)
    return chunks
```

Indexing Phase of Embeddings

Indexing is the process of converting chunks into vectors and storing them in a vector database.

📌 Why index?
- Allows a quick search in a large number of documents.
- Facilitates the efficient retrieval of relevant information.
- Optimizes access to similar data in terms of content. Ultra-fast.



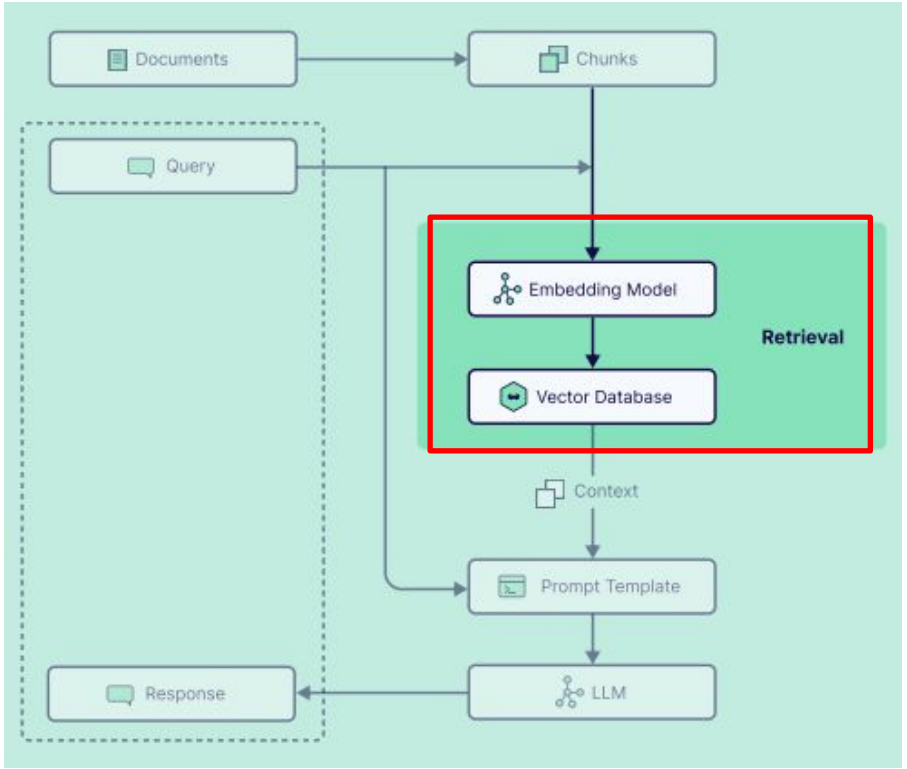
Indexing Phase of Embeddings

Indexing Steps

- 1. Creating embeddings
Each chunk is transformed into a digital vector by an embedding model.

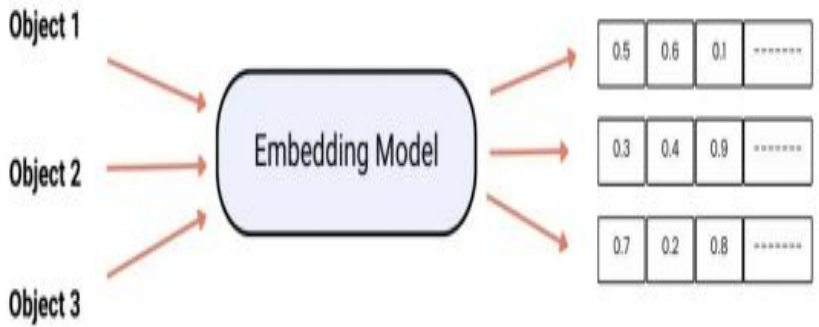
- 2. Storage in a vector base
Embeddings are saved in a database like FAISS, ChromaDB or Pinecone.

- 3. Indexing for Quick Find
An index is built to allow for ultra-fast similarity search.



Indexing Phase of Embeddings

1.1 Embedding Model:



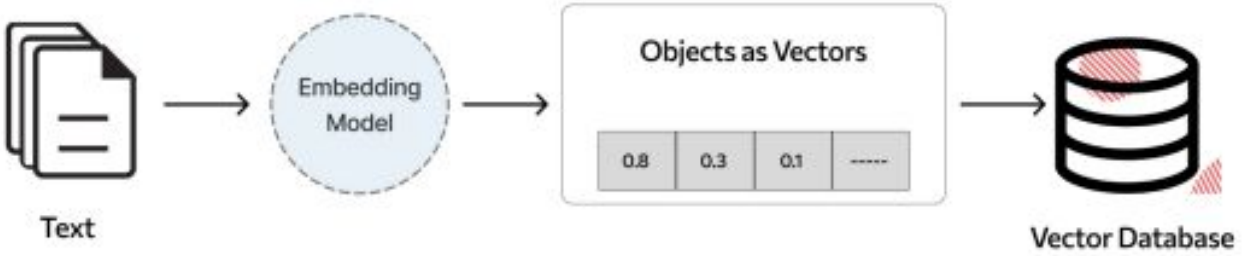
The embedding model converts text into a numerical representation (vectors) that captures the semantic meaning of words and sentences.

👉 Why? This transformation makes it possible to effectively compare texts by measuring their similarity in a vector space.

Indexing Phase of Embeddings

1.2 Vector Database:

A vector database stores and indexes the embeddings of documents to allow a quick search for relevant information.



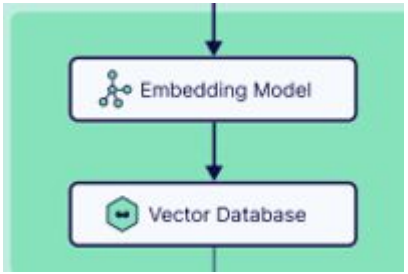
👉 Pourquoi ? Lorsqu'un utilisateur pose une question, le pipeline recherche les documents les plus proches en termes de similarité sémantique.


Pinecone
Pinecone
Milvus
Milvus
Weaviate
Weaviate
VECTOR DATABASES
FAISS
FAISS
Qdrant
qdrant
KDnuggeets
KD

Indexing Phase of Embeddings

3. Creation of embeddings + Storage in a vector base

```python
def get_vector_store(text_chunks):
    embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
    vector_store=FAISS.from_texts(text_chunks,embedding=embeddings)
    vector_store.save_local("vector_store")
```



1.3 Prompt Template:

The prompt template is a structured template used to integrate the information retrieved in a query intended for the LLM.

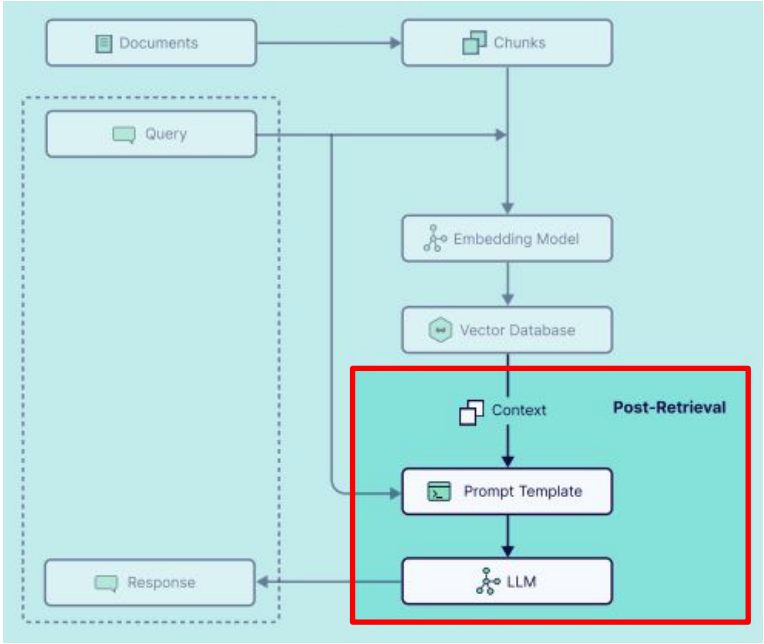
👉 Why? This allows you to enrich the model with relevant context in order to obtain a more precise answer.

```
Contexte : {documents_récupérés}

Question : {question_utilisateur}

Réponse :
```

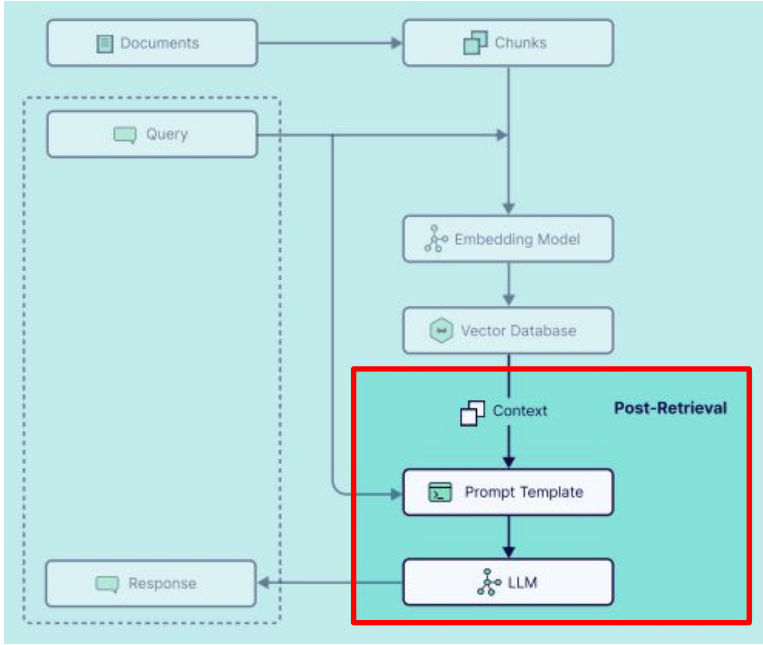
Ce format guide le modèle en lui fournissant les documents récupérés avant de générer une réponse.



1.4 Modèle de langage génératif (Generative LLM)

The LLM is the heart of the RAG system. It receives the rich prompt with external documents and generates a response.

👉 Why? Unlike traditional LLMs that generate responses based solely on their training, here the model is supported by external information.



```python
def get_conversational_chain():
    prompt_template="""answer the question as detailed as possible from the provided context,
    make sure to provide all the details,if the answer is not in the provided context,
    just say "I am sorry, answer is not avaible in this context" don't provide a wrong answer\n\n
    Context: {context} \n\n
    Quetion: {question} \n\n"""

    model=ChatGoogleGenerativeAI(model="gemini-2.5-flash")
    prompt=PromptTemplate( template=prompt_template,input_variables=["context","question"])
    chain=load_qa_chain(llm=model, prompt=prompt)
    return chain

def user_input(user_question):
    embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
    new_db=FAISS.load_local("vector_store",embeddings,allow_dangerous_deserialization=True)#
    docs = new_db.similarity_search(user_question)
    chain=get_conversational_chain()
    response = chain(
        {"input_documents":docs, "question": user_question}
        , return_only_outputs=True)

    print(response)
    st.write("", response["output_text"])
```